Supervised Learning Report FAQ
CS7641: Machine Learning
Summer 2026

1 Frequently Asked Questions (FAQ)

This FAQ is intentionally extensive. Everything required for the assignment is already stated in the project description, but each term many of the same questions come up as students begin sampling data, running models, building figures, and writing the report. This FAQ is meant to help you as you move through the experimentation and writing process.

You do not need to absorb this entire document at once. There is a lot of information at the beginning of the assignment, and that can feel overwhelming before you have started working. Use this FAQ as a guide while you work. When you are sampling the data, read the sampling section. When you are building curves, read the figures section. When you are writing the discussion, read the discussion and writing sections.

The assignment is open-ended in analysis, but the expectations are concrete. The central idea is simple, you are not just running algorithms. You are building a reproducible supervised learning study, using evidence to explain how different model families behave on the same multiclass classification problem.

How to use this FAQ

This FAQ is organized around the order in which most students encounter problems. Use it as a reference while you work rather than as something to memorize at the beginning.

When you are working on...	Read these sections
Understanding the assignment	Project Scope
Creating the dataset	Sampling and Data Setup
Deciding what to inspect in the data	Exploratory Data Analysis
Building the evaluation workflow	Metrics and Evaluation
Creating plots and tables	Required Figures and Tables
Running model experiments	Algorithm-Specific Guidance
Writing the analysis	Discussion and Conclusion; Writing and Time Planning
Final submission checks	Reproducibility and Submission; Page Limit and Writing Style; Quick Self-Check

Why are there minimum numbers in the assignment?

Some requirements specify minimum numbers, such as at least three reasonable hyperparameter values, at least four training sizes for learning curves, at least two SVM kernels, and two neural-network implementations. These minimums are not meant to encourage mechanical box-checking. They exist so that your analysis has enough evidence to support a claim.

For example, testing only one value of k, one tree depth, or one learning rate does not show whether the model is sensitive to that choice. Testing at least three reasonable values gives you a small but interpretable comparison such as a low-complexity setting, a middle setting, and a higher-complexity setting. This is often enough to reveal underfitting, overfitting, saturation, or hyperparameter sensitivity.

Similarly, using at least four training sizes in a learning curve helps distinguish a noisy trend from a real learning pattern. Comparing at least two SVM kernels helps separate the effect of margin-based classification from the

effect of nonlinear feature mappings. The goal is not to run the largest possible search. The goal is to create enough evidence to justify your final model choices and explain model behavior.

Requirement	Why this minimum exists
At least three hyperparameter values	Provides enough evidence to discuss sensitivity, underfitting, overfitting, or saturation.
At least four training sizes	Makes learning-curve trends more interpretable than a two-point comparison.
At least two SVM kernels	Allows comparison between linear and nonlinear decision boundaries.
Two NN implementations	Supports comparison of optimization behavior and implementation differences under the SGD-only constraint.
Per-class metrics	Prevents aggregate scores from hiding poor minority-class behavior.

2 Project Scope

Q: What exactly is the task?

You must perform **multiclass classification** using the Forest Cover Type dataset, also known as Covertype. The target is `Cover_Type`, and the classes are 1 through 7. You will create an approximately 20,000-instance working dataset from the full dataset and compare supervised learning algorithms using EDA, tuning, learning curves, model-complexity curves, runtime, and multiclass error patterns.

Item	Requirement
Dataset	Forest Cover Type / Covertype
Target	`Cover_Type`
Task	Seven-class multiclass classification
Working sample	Approximately 20,000 instances sampled from the full dataset
Data access	UCI Machine Learning Repository, Canvas, or `sklearn.datasets.fetch_covtype`
Main goal	Explain algorithm behavior using evidence, not only final scores

Q: Can I change the task?

No. You must classify `Cover_Type` as a seven-class multiclass classification problem. Do not collapse the target into a binary classification problem, do not treat the target as regression, and do not substitute another dataset.

Q: Do I need to use the full dataset?

No. The full dataset contains more than half a million instances. You are required to create an approximately **20,000-instance working dataset** and justify your sampling strategy. The point is not to avoid computation blindly. The point is to make a deliberate, reproducible sampling choice and then discuss how that choice affects class balance, minority classes, runtime, and confidence in your results.

Q: What are the required algorithm families?

You must evaluate all four required algorithm families on your sampled Covertype dataset. Neural networks require two implementations: one scikit-learn model and one PyTorch model.

Algorithm family	Minimum expectation
Decision Trees	Use pruning or regularization, such as depth limits, leaf constraints, or cost-complexity pruning.
kNN	Use appropriate scaling, compare meaningful k values, and discuss prediction-time cost.
SVM	Use at least two kernels, such as linear and RBF, and tune C and γ where relevant.
Neural Networks	Train one scikit-learn `MLPClassifier` and one compact PyTorch MLP using SGD only.

3 Sampling and Data Setup

Q: How should I create the approximately 20,000-instance dataset?

The default expectation is **stratified sampling**. The target classes are imbalanced, so stratification helps preserve the class proportions of the full dataset. Your report should state the full-data size, sample size, full-data class distribution, sampled-data class distribution, random seed, sampling method, and why that sampling method is appropriate.

A strong sampling discussion does not simply say "I sampled 20,000 rows." It explains whether the sample preserves the important properties of the original dataset, especially the target distribution.

Q: Is stratified sampling required?

Stratified sampling is the default expectation because the dataset is imbalanced. A simple random sample may be acceptable only if you show that all seven classes are preserved, the sampled class proportions are reasonably close to the full-data proportions, minority classes still have enough examples to support evaluation, and the sampling procedure is reproducible. If your random sample noticeably changes the class distribution, you should either justify why that is acceptable or switch to stratified sampling.

Q: Can I use the first 20,000 rows?

No, not without a compelling justification. Using the first 20,000 rows can introduce ordering effects. If the data are sorted or partially structured by class, location, source, or collection order, taking the first rows may produce a biased sample. Use a randomized or stratified procedure instead. If you choose a nonrandom approach, explain why it is defensible and verify that it does not distort the target distribution.

Q: What should I report about the class distribution?

Report both counts and proportions for the full dataset and your sampled dataset. Your report should make clear which classes dominate the dataset, which classes are rare, whether the 20,000-instance sample preserves the full-data distribution, whether the rarest classes are represented well enough for meaningful evaluation, and how imbalance affects Accuracy, Macro-F1, balanced accuracy, and confusion-matrix interpretation.

4 Exploratory Data Analysis

Q: What is the purpose of EDA in this assignment?

EDA is not filler. EDA should set up the modeling story. It should help justify your sampling strategy, metric choices, preprocessing steps, algorithm expectations, likely error patterns, and hypothesis. The best EDA sections do not show every possible plot. They show the evidence needed to motivate the later experiments.

Q: What basic EDA is required?

At minimum, you should examine dataset shape, target classes, feature names, feature groups, feature types, missingness or absence of missingness, full-data class distribution, sampled-data class distribution, numeric feature ranges, binary feature groups, obvious scale differences, and any dataset properties that may affect model behavior.

The main report does not need every table or plot. It does need enough EDA evidence to stand alone. Extended EDA can go in the reproducibility sheet.

Q: What should I look for in the feature types?

Covertype contains continuous cartographic variables, binary wilderness-area indicators, binary soil-type indicators, and the target variable `Cover_Type`. This matters because algorithms respond differently to feature

types. kNN and SVM are sensitive to feature scale. Neural networks usually benefit from scaled numeric inputs. Decision Trees do not require scaling in the same way. Binary indicator features can affect distance-based methods if they are mixed with unscaled continuous features.

Q: What should I look for in feature scales and ranges?

Report whether continuous features have very different ranges. One feature might span thousands of units while another spans a much smaller range. This matters because kNN distances, SVM margins and kernels, and neural-network optimization are all affected by feature scale. Decision Trees are generally less sensitive to monotonic scaling because they split on thresholds.

You do not need to include a giant table of every statistic in the main report. A compact summary of ranges, scale differences, and implications is better.

Q: What should I look for in the target distribution?

The target distribution is central to this assignment. Look for majority classes, minority classes, whether the 20,000-instance sample preserves the full-data distribution, whether rare classes may have unstable precision or recall estimates, and whether Accuracy might hide poor minority-class performance. This EDA should directly motivate Macro-F1, balanced accuracy, a confusion matrix, and per-class precision/recall/F1.

Q: What should I look for in class-conditional feature patterns?

Class-conditional EDA asks whether features look different across cover types. You might compare key continuous features by `Cover_Type`, examine feature means or medians across classes, look for strong class overlap, or identify features that separate one class better than others.

These patterns can guide expectations. Nonlinear separation may favor RBF SVM or neural networks. Threshold-like separation may make Decision Trees competitive. Strong class overlap may explain confusion between cover types. Sparse minority classes may make kNN sensitive to k and sampling.

Q: Do I need correlation plots?

Correlation plots can be useful, but they are not required for their own sake. Use correlation or feature-association analysis if it helps answer a modeling question, such as whether continuous features are redundant, whether feature groups are related, or whether correlated inputs may affect distance-based models or neural-network training. Do not include a large unreadable correlation heatmap unless you interpret it.

Q: What EDA should go in the main report versus the reproducibility sheet?

The main report should include the EDA evidence needed to support your analysis. This usually means the class distribution for full and sampled data, key feature-type and feature-scale observations, one or two findings that motivate preprocessing or hypotheses, evidence relevant to imbalance and metric choice, and a brief connection between EDA and expected model behavior.

The reproducibility sheet can contain longer summary tables, additional plots, code or commands used to generate EDA, the full sampling procedure, exact random seeds, and supporting details that would not fit in the 8-page main report. Do not move all interpretation to the reproducibility sheet. The main report must stand alone.

Q: How should EDA connect to my hypothesis?

Your hypothesis should be motivated by what you observed. If feature scales differ widely, you might hypothesize that scaled kNN and SVM will improve substantially over unscaled variants. If class imbalance is strong, you might hypothesize that Accuracy will favor majority-class performance while Macro-F1 exposes minority-class weaknesses. If class-conditional plots suggest nonlinear separation, you might hypothesize that RBF SVM or an MLP will outperform linear SVM. If some classes appear separable by threshold-like rules, you might hypothesize that a regularized Decision Tree will be competitive.

5 Metrics and Evaluation

Q: What metrics are required?

You must report Accuracy, Macro-F1, balanced accuracy, a confusion matrix, and per-class precision, recall, and F1 discussion. These metrics are required because the task is multiclass and imbalanced.

Metric	Why it matters
Accuracy	Summarizes overall correctness but can be dominated by majority classes.
Macro-F1	Weights classes equally and helps reveal minority-class weaknesses.
Balanced accuracy	Averages recall across classes and reduces majority-class dominance.
Confusion matrix	Shows which cover types are confused with one another.
Per-class precision/recall/F1	Shows whether specific classes are being ignored, overpredicted, or confused.

Q: Why is Accuracy not enough?

Accuracy can be dominated by the majority classes. A model can achieve strong Accuracy while performing poorly on rare cover types. Macro-F1 gives each class equal weight, so it is more sensitive to minority-class behavior. Balanced accuracy averages recall across classes, which helps identify whether the model performs broadly or mostly succeeds on common classes. Your report should discuss situations where Accuracy, Macro-F1, and balanced accuracy tell different stories.

Q: How should I split the data?

Use one held-out test set for final evaluation. Do not tune on the test set. A standard workflow is to create the 20,000-instance working dataset, split once into training and test sets, tune only on the training data using cross-validation or a validation split, train the final model using the chosen hyperparameters, and evaluate once on the held-out test set. Use stratified splitting unless you provide a strong reason not to.

Q: Do I need cross-validation?

You must use either cross-validation or an explicit validation split for hyperparameter tuning. Cross-validation is recommended when feasible. A single train/validation split is acceptable when runtime is restrictive, especially for expensive models. If you use a single validation split, explain why and make sure the test set remains untouched until final evaluation.

Q: How do I avoid leakage?

Leakage occurs when information from validation or test data influences training or model selection. Common risks include fitting scalers before splitting, choosing hyperparameters based on test performance, repeatedly checking the test set during tuning, using preprocessing steps fit on all data, or allowing the sampled test set to influence feature selection. Split first, fit preprocessing only on training data, use pipelines where possible, tune only on training folds or validation data, and evaluate the test set once at the end.

Q: What is the Leakage / Evaluation Debug rule?

If your results are suspiciously strong or implausibly perfect, include a short **Leakage / Evaluation Debug** subsection. Useful diagnostics include a dummy classifier baseline, majority-class baseline, shuffled-label test where performance should collapse, confirmation that scalers were fit on training data only, confirmation that the test set was not used for tuning, and a check that target labels were not accidentally included as features. Even when your results are not suspicious, a simple baseline is often useful because it helps interpret how meaningful your model improvements are.

6 Required Figures and Tables

Q: What figures and tables are required?

Your report should include EDA summary evidence, learning curves, model-complexity curves, a runtime table, a final metric table, and confusion-matrix or class-level error analysis. You are encouraged to combine plots into compact multi-panel figures to fit the 8-page limit.

Q: What should a learning curve show?

A learning curve shows how performance changes as the model receives more training data. Use training size on the x-axis and a relevant metric, such as Macro-F1, balanced accuracy, or Accuracy, on the y-axis. Include training and validation scores, and use at least four training sizes unless computationally justified.

Interpret the curve rather than only displaying it. Low and close training/validation scores suggest underfitting. High training score with much lower validation score suggests overfitting. A validation curve that is still rising at the largest training size suggests more data may help. Early saturation suggests that more data alone may not solve the issue without changing capacity, features, or hyperparameters.

Q: What should a model-complexity curve show?

A model-complexity curve isolates one hyperparameter and shows how validation performance changes. Use one hyperparameter on the x-axis, validation performance on the y-axis, and at least three reasonable values unless computationally justified. Three values is the minimum because one value gives no comparison and two values often only show direction. Three values can begin to show whether performance improves, degrades, saturates, or has a useful middle region. Examples include `max_depth` or `ccp_alpha` for DT, k for kNN, C or γ for SVM, and width, depth, learning rate, or regularization for NN. The curve should support the final model choice.

Do not vary many settings at once unless the plot remains interpretable. The purpose is explanation, not just optimization.

Q: What should the neural-network epoch curve show?

For neural networks, include an epoch-based curve showing training and validation loss or metric over epochs. Use this curve to discuss whether training loss decreased smoothly, whether validation performance improved and then degraded, whether the model appeared unstable, whether early stopping helped, whether the learning rate appeared too high or too low, and whether the scikit-learn and PyTorch implementations behaved differently.

Q: What should the runtime table include?

Report wall-clock fit time and predict time for the final model configurations. Include a brief hardware note and whether a GPU was used for neural networks. The runtime discussion should explain trade-offs. kNN may have cheap fitting but expensive prediction, kernel SVM may become expensive as training size grows, Decision Trees may train quickly and remain interpretable, and neural networks may depend strongly on epochs, batch size, learning rate, and implementation details.

Q: How do I fit everything in 8 pages?

Use compact figures and avoid redundancy. Multi-panel learning-curve figures, multi-panel model-complexity figures, one final metric table, one runtime table, and one confusion-matrix or class-level summary figure are usually more effective than many small plots. Extended EDA belongs in the reproducibility sheet. Do not spend space restating legends. Every figure should have a takeaway.

7 Algorithm-Specific Guidance

Q: What should I do for Decision Trees?

Use at least one complexity control, such as `max_depth`, `min_samples_leaf`, `min_samples_split`, or `ccp_alpha`. Your analysis should discuss how tree depth or pruning affected bias/variance, whether the tree overfit without regularization, the final depth or number of leaves, runtime compared with other models, and whether threshold-based splits seem appropriate for the dataset.

Look to EDA feature distributions for signs that threshold splits may work well. The DT complexity curve can show where the tree begins to overfit, and the learning curve can show whether more data reduces variance.

Q: What should I do for kNN?

Use appropriate scaling and test meaningfully different k values. At least three distinct values are expected unless you justify otherwise. Reasonable k choices should include small, medium, and larger neighborhoods. For example, $k = 3$, $k = 11$, and $k = 31$ might be reasonable, but the exact values should make sense for your sample size and results.

Your analysis should discuss distance metric, weighting choice, effect of k, sensitivity to feature scaling, prediction-time cost, and whether minority classes are helped or hurt by different k values. Feature-scale EDA explains why scaling matters, class overlap can explain misclassification among similar cover types, the k-complexity curve can show bias/variance behavior, and runtime results can reveal prediction-time cost.

Q: What should I do for SVM?

Evaluate at least two kernels, such as linear and RBF. Tune C for linear and nonlinear kernels, and tune γ for RBF or other nonlinear kernels where relevant. For computationally expensive SVM experiments, a reduced grid or validation split is acceptable if you justify it with runtime evidence.

Your analysis should discuss why scaling is required, how the kernels differ, whether nonlinear kernels improved performance, how C and γ affected performance, runtime trade-offs, and whether the performance gain justified the cost. Feature-scale EDA explains scaling needs, class overlap may explain kernel differences, the SVM complexity curve can show sensitivity to C or γ, and runtime results can explain whether the kernel choice is practical.

Q: What exactly does SGD-only mean for neural networks?

For neural networks, SGD-only means no momentum, no Nesterov momentum, no Adam, no AdamW, no RMSprop, no Adagrad, and no other adaptive optimizer. For scikit-learn `MLPClassifier`, use `solver='sgd'`, `momentum=0`, and `nesterovs_momentum=False`. For PyTorch, use plain `torch.optim.SGD` without momentum.

Q: What should I compare between scikit-learn and PyTorch NNs?

You should compare more than final scores. Useful comparison points include optimization stability, epoch curves, sensitivity to learning rate, runtime, implementation flexibility, regularization choices, early stopping behavior, and final class-level performance. Epoch curves show optimization stability, learning curves show whether more data may help, complexity curves show sensitivity to architecture or learning rate, and runtime results show whether the implementation is practical.

Q: What tuning strategy should I use?

A full grid search is not required and often is not the best use of compute. It can become expensive quickly, especially for SVMs and neural networks. A better strategy is usually to start with a small number of reasonable values, inspect the results, and then refine the range if the best setting is at the edge or if the curve suggests sensitivity.

At minimum, your experiments should include enough variation to support interpretation. For a scalar hyperparameter, this usually means at least three reasonable values. These should not be tiny variations around the same setting unless you justify why that narrow range matters. For example, $k = 5, 6, 7$ usually does not

reveal much about kNN behavior, while a small, medium, and larger neighborhood can show how locality and smoothing affect performance.

Useful strategies include coarse-to-fine search, random search over sensible ranges, log-scaled sweeps for C, γ, learning rate, and regularization, and small targeted experiments motivated by EDA or early validation results. For example, rather than testing every combination of many parameters, you might first compare broad C values for SVM, identify a promising region, and then run a narrower follow-up. For neural networks, you might first stabilize learning rate and batch size before comparing architecture depth or width.

Regardless of tuning method, you still need interpretable model-complexity curves. The curve should isolate one important hyperparameter and show how it affects validation performance.

8 Discussion and Conclusion

Q: What should the discussion section do?

The discussion should explain what the full set of experiments reveals. It should not simply repeat model-by-model results. A strong discussion compares at least three algorithm families, explains trade-offs between performance and runtime, discusses class-level behavior and minority-class errors, uses learning curves and model-complexity curves to explain model behavior, and identifies one limitation and one evidence-based improvement or follow-up experiment.

Q: What does cross-model synthesis mean?

Cross-model synthesis means comparing algorithms to explain patterns.

Weak example:

> The SVM had the best Macro-F1, kNN was second, and Decision Tree was third.

Stronger example:

> The RBF SVM improved Macro-F1 relative to the linear SVM and Decision Tree, suggesting nonlinear boundaries mattered. However, its fit time was substantially higher. kNN performed competitively on common classes but struggled with rare classes, possibly because minority examples were sparse in local neighborhoods. The Decision Tree trained quickly but showed a larger train-validation gap at high depth, consistent with overfitting.

The stronger version explains why the results happened and what trade-offs they imply.

Q: What questions should my discussion answer?

Your discussion should answer questions such as: Which model performed best overall, and does that answer change across Accuracy, Macro-F1, and balanced accuracy? Which model was most efficient, and was the performance gain worth the runtime cost? Which model had the best minority-class behavior? Which cover types were most often confused? Did high Accuracy hide poor minority-class behavior? Did learning curves suggest high bias, high variance, saturation, or benefit from more data? What limitation most affects confidence in your final conclusion? What follow-up experiment is directly motivated by your evidence?

Q: What does class-level analysis mean?

Class-level analysis means looking beyond aggregate metrics. You should discuss which classes had the highest and lowest recall, which classes were commonly confused, whether minority classes performed worse than majority classes, whether any model improved minority-class performance, and whether aggregate Accuracy hides important class-specific failures. Use the confusion matrix and per-class precision/recall/F1 to support these claims.

Q: What should limitations and improvements look like?

Limitations should be concrete and tied to your study. Good limitations might include the 20,000-instance sample, unstable rare-class estimates, a limited SVM grid due to runtime, the SGD-only neural-network constraint, not testing class weighting or resampling, or limited feature engineering.

Good improvements follow directly from evidence. For example, test class weighting if minority-class recall was poor. Expand the SVM C/γ sweep if the best model was at the edge of the tested range. Increase minority-class representation if Macro-F1 appears unstable. Try a learning-rate schedule for SGD if epoch curves showed instability. Weak improvements such as "use more data," "tune more," or "try more models" do not add much unless you explain what failure mode the improvement addresses.

9 Writing and Time Planning

Q: When should I start writing?

Start writing before all experiments are finished. The report will be much stronger if the writing develops alongside the experiments. Write the dataset, sampling, EDA, metric justification, hypothesis, evaluation protocol, and preprocessing workflow early. Then add algorithm-specific interpretations as each model family finishes. Save the final cross-model discussion for after you can compare across algorithms.

Q: How should I plan my time?

Chunk the work by dependency rather than by model alone. First, complete the data foundation: loading, sampling, EDA, class distributions, split strategy, preprocessing, metrics, and baseline checks. This foundation should be stable before you run many experiments.

Next, you should focus on building the experiment harness using the faster models. Decision Trees and kNN are useful early because they can expose problems with metrics, splitting, scaling, runtime measurement, and plotting. After that, move to SVM with smaller trial runs first so you can understand computational cost before committing to larger searches. Neural networks should come after the data pipeline and evaluation code are stable, since debugging architecture, learning rate, epoch curves, and implementation differences is much easier when the rest of the workflow already works.

Reserve a separate writing and synthesis chunk. Final tables and figures should be built before the discussion, but the discussion should not be left until the last minute. You need time to compare models, interpret class-level errors, connect curves to bias/variance behavior, and write limitations that follow from the evidence. Also reserve a final reproducibility chunk for checking the Overleaf link, GitHub commit hash, run commands, seeds, citations, AI statement, and PDF formatting. This last step is often overlooked and rushed.

Q: What should I write first?

Write the parts that do not depend on final results first: dataset description, sampling method, EDA summary, metric justification, hypothesis, evaluation protocol, and preprocessing workflow. These sections create the logic for the experiments. If you cannot explain these clearly, the results section will usually become a list of numbers rather than an analysis.

Q: How do I avoid writing a checklist?

A checklist reports what you did. A technical paper explains why the choices matter and what the results mean.

Weak:

> I ran DT, kNN, SVM, and NN. The SVM had the best Macro-F1. The DT was fastest. The NN was slower.

Stronger:

> The RBF SVM achieved the strongest Macro-F1, suggesting that nonlinear boundaries were useful for separating cover types with overlapping feature distributions. However, this came with the

highest training cost among the non-neural models. The Decision Tree trained quickly and was easier to interpret, but its learning curve showed a persistent train-validation gap at larger depths, suggesting variance that pruning only partially controlled.

Use the stronger style. Claims should connect results, evidence, and course concepts.

Q: How should I revise the report after the first draft?

After the first draft, check whether every figure has a takeaway, every table supports a claim, every model has an explanation, minority classes are discussed, models are compared directly, curves are used in the discussion, limitations and improvements are specific, redundant plots are removed, and all results can be reproduced from the submitted commit.

10 Reproducibility and Submission

Q: Do I need to use the IEEE conference template?

Yes. The main report must use the official IEEE conference paper template, such as:

```
\documentclass[conference]{IEEEtran}
```

A generic article template or custom report format is not acceptable for the main report. The reproducibility sheet does not need to use the IEEE template, but it must be clear and readable.

Q: Do I need to use Georgia Tech / Enterprise Overleaf?

Yes. The main report must be created using your Georgia Tech / Enterprise Overleaf account. You must provide a read-only Overleaf link. Do not send email invitations.

Q: Do I need to use GT Enterprise GitHub?

Yes. Course-related code must be stored in GT Enterprise GitHub. Your reproducibility sheet must include a single commit hash from the final version of your code. Personal GitHub repositories are not acceptable for this assignment.

Q: What must be included in the reproducibility PDF?

Your `REPRO_SL_{GTusername}.pdf` must include the read-only Overleaf link, Git commit hash, exact run commands, environment details, data access instructions, random seeds, sampling procedure, full-data and sampled-data EDA summaries, and enough information to reproduce the report figures and tables.

Q: Do I need outside sources?

Yes. You need at least **two peer-reviewed references** beyond course materials. These should be used to support your methodological choices or interpretation, not merely listed in the references section.

Primary sources are strongly preferred. In this context, a primary source is usually an original peer-reviewed paper that introduces, evaluates, or studies a method, metric, algorithm, or evaluation issue directly. For example, a paper on imbalanced classification metrics, pruning, support vector machines, cross-validation, leakage, or neural-network optimization is more appropriate than a broad summary article.

Avoid relying on textbooks for this requirement. Textbooks can be useful for learning background material, but they are usually too general for supporting specific experimental choices in this report. Your outside sources should help justify decisions such as why Macro-F1 is appropriate for imbalanced multiclass classification, why leakage-safe preprocessing matters, why a pruning parameter controls tree complexity, or how kernel choice affects SVM behavior.

Peer review matters. Avoid citing arXiv papers, preprint servers, blog posts, documentation pages, tutorials, lecture notes, Wikipedia, Medium posts, or vendor documentation as one of your required peer-reviewed references. These may be useful for implementation help or background reading, but they should not replace the required scholarly sources.

Use **IEEE citation style**. A strong report uses sources in the body of the paper to support claims. For example, do not only place two papers in the bibliography. Cite them where they help justify a metric, evaluation protocol, leakage control, algorithmic behavior, or interpretation of results.

Q: Can I use AI tools?

Yes, with disclosure. You may use AI tools for brainstorming, outlining, grammar edits, code generation, code refactoring, and debugging. However, your submitted analysis must be your own. You are responsible for understanding and verifying all code, figures, and text.

Your report must include an AI Use Statement that identifies which tools you used, what they helped with, and that you reviewed and understood all assisted content.

11 Page Limit and Writing Style

Q: How do I fit the assignment into 8 IEEE pages?

Plan your figures carefully. Use compact multi-panel figures, one final metric table, one runtime table, and one confusion-matrix or class-level summary figure. Move extended EDA to the reproducibility sheet. Do not waste space restating legends. Every figure and table should have a takeaway.

Q: Can I use bullets in the report?

Yes, but sparingly. Bullets are useful for listing hyperparameters, summarizing preprocessing steps, describing an experimental protocol, or making compact reproducibility statements. Your analysis and conclusions should be written in paragraphs. A report that is mostly bullet points will read like a checklist rather than a technical paper.

Q: What does "interpret, do not summarize" mean?

A summary says what happened. An interpretation explains why it matters.

Weak:

> The RBF SVM had the highest Macro-F1.

Stronger:

> The RBF SVM had the highest Macro-F1, which suggests that nonlinear decision boundaries helped separate cover types that overlapped in the original feature space. However, the runtime table shows that this improvement came at a substantial computational cost compared with the Decision Tree.

Your report should prefer the second style.

12 Quick Self-Check

Before submitting, use this checklist to catch common issues.

Category	Check
Task	The report uses Covertype and predicts `Cover_Type` as classes 1 through 7.
Sampling	The report uses an approximately 20,000-instance sample and explains how it was created.
Class distribution	The report includes both full-data and sampled-data class distributions.
EDA	The EDA motivates sampling, metrics, preprocessing, model expectations, and the hypothesis.
Metrics	The report includes Accuracy, Macro-F1, balanced accuracy, a confusion matrix, and per-class precision/recall/F1 discussion.
Evaluation	Hyperparameters are tuned only on training data, and the held-out test set is used only for final evaluation.
Leakage control	Preprocessing is fit only on training data or training folds.
Algorithms	The report includes DT, kNN, SVM with at least two kernels, scikit-learn MLP, and PyTorch MLP.
NN optimizer	Neural networks use SGD only, with no momentum, Nesterov, Adam, RMSprop, Adagrad, or other adaptive optimizers.
Curves	The report includes learning curves and model-complexity curves with interpretation.
Runtime	The report includes fit and predict time with a hardware note.
Discussion	The discussion compares models, explains trade-offs, analyzes class-level behavior, uses curves to explain behavior, and identifies limitations and improvements.
Format	The main report uses the IEEE conference template.
Overleaf	The report includes a read-only Georgia Tech / Enterprise Overleaf link.
GitHub	The reproducibility sheet includes a GT Enterprise GitHub commit hash.
Sources	The report cites at least two peer-reviewed sources in IEEE style.
AI statement	The report includes an AI Use Statement.
Reproducibility	The reproducibility PDF includes commands, seeds, sampling details, EDA summaries, data access information, and enough detail to reproduce figures and tables.